Clustering Performance VS Data Size Report

Final Submission

Nevaeh Zumbrun, Jacob Batton, Patriya Murray

Problem Overview and Methods:

We want to compare clustering algorithms with their efficiency, accuracy, and scalability.

Objective / Research Questions:

What clustering algorithm produces the most accurate clusters? What algorithm has the best runtime with increasing data sizes? What algorithm costs the least amount of memory? With all these questions in mind, what is the most appropriate algorithm as data size increases? We chose these questions because we want to best measure efficiency (runtime), accuracy (ability to identify clusters), and scalability (memory cost).

Description of Experiment(s):

Implementation programming language: Python

Input description (size, type: patterned/random/real or artificial dataset): Small algorithm: Iris dataset, 150 elements, random, real dataset. Bigger algorithms: Synthetic, 200-20000 elements, random, artificial dataset.

Number or repeats: 10 for each algorithm

Characteristics of the computer you used to generate results: Every person used their own personal computer for this project, thus each computer is going to be different.

Reference method (such as brute force): Since we are using clustering algorithms, we didn't use a typical reference method. We used the silhouette score to measure accuracy and compared that metric with all the other algorithms.

Performance metrics (execution time, counts of basic operations, how close your optimal solution is to the optimal solution): The performance metrics we used were runtime, silhouette score and memory (MB). Each of these varied with different datasets and algorithms.

Make clear what you did: We wrote four clustering algorithms (K-means, DBSCAN, Hierarchical clustering, and Firefly) in python. We used a graph library to generate graphs based off of the runtime, silhouette score, and memory (MB) of each algorithm based on a scaling dataset. Afterwards, we compared the algorithms together based on those metrics to conclude the best clustering algorithm.

Results:

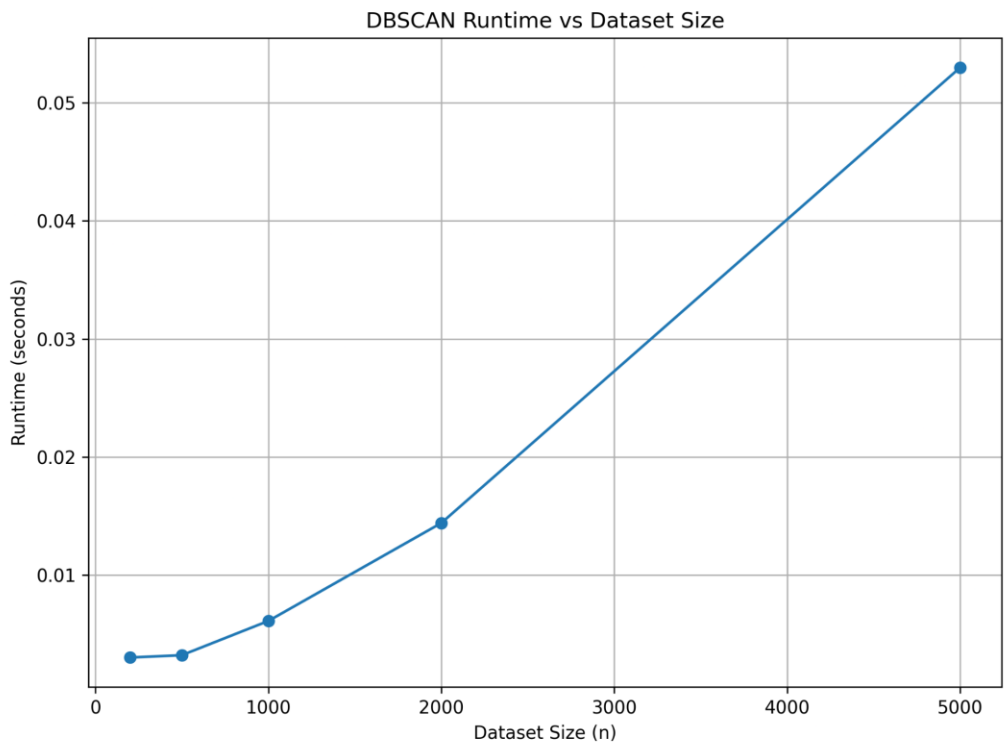
DBSCAN:

DBSCAN was evaluated with both the synthetic and iris datasets to analyze memory usage, runtime, accuracy, and how increasing the data size affects these metrics.

1. Runtime

As anticipated, the runtime of DBSCAN increased as the dataset size (n). We found that runtime consistently grows larger as the dataset size (n) increases.

DBSCAN has slower performance than other algorithms when it comes to handling large datasets, making it less efficient when runtime is a primary concern.



2. Accuracy (Cluster Quality)

For measuring cluster quality, we used the silhouette score. For the iris data set, we found that the parameter combination of **eps = 0.5** and **min_samples = 5** produced the best clusters on average. This achieved a silhouette score of approximately .656. This indicates that DBSCAN performs well on smaller datasets. For consistency the same **(eps = 0.5, min_samples = 5)** were used on our synthetic dataset, we observed that as the dataset size increased, DBSCAN performed very well and produced silhouette scores greater than 0.5, reaching up to 0.85. This demonstrates that DBSCAN can maintain high accuracy even with larger datasets. The main factors that appear to affect DBSCAN silhouette scores is the choice of eps and min_samples parameters. However the synthetic data may have varying cluster densities and separations, which is why the silhouette score does not follow a consistent trend. This means that DBSCAN's performance is affected more by data distribution than data size alone.

Preview | Code | Blame 6 lines (6 loc) · 393 Bytes

Search this file

algorithm	dataset	n	eps	min_samples	runtime_seconds	memory_mb	num_clusters	noise_points	silhouette_score
DBSCAN	Synthetic	200	0.5	5	0.0009	0.0258	5.0	64.0	0.6139
DBSCAN	Synthetic	500	0.5	5	0.0018	0.0137	4.0	49.0	0.6721
DBSCAN	Synthetic	1000	0.5	5	0.0041	0.0277	3.0	35.0	0.8522
DBSCAN	Synthetic	2000	0.5	5	0.0088	0.1305	4.0	38.0	0.6728

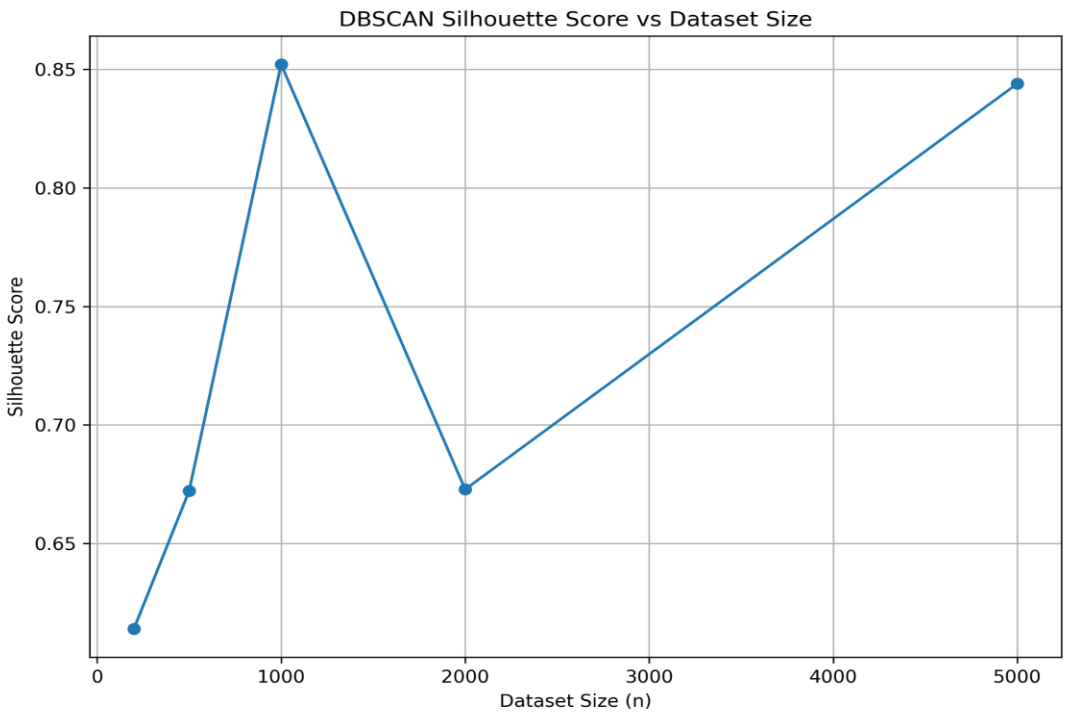
DBSCAN using synthetic data as the dataset size (n) increases

Preview | Code | Blame 13 lines (13 loc) · 722 Bytes

Search this file

algorithm	dataset	n	eps	min_samples	runtime_seconds	memory_mb	num_clusters	noise_points	silhouette_score
DBSCAN	Iris	150	0.3	3	0.0015	0.0461	10.0	84.0	0.3974
DBSCAN	Iris	150	0.3	5	0.0008	0.0078	3.0	120.0	0.6302
DBSCAN	Iris	150	0.3	7	0.0008	0.0109	2.0	129.0	0.6276
DBSCAN	Iris	150	0.5	3	0.0038	0.0055	7.0	17.0	0.2591
DBSCAN	Iris	150	0.5	5	0.0008	0.0062	2.0	34.0	0.6559

DBSCAN using the iris dataset across different parameters

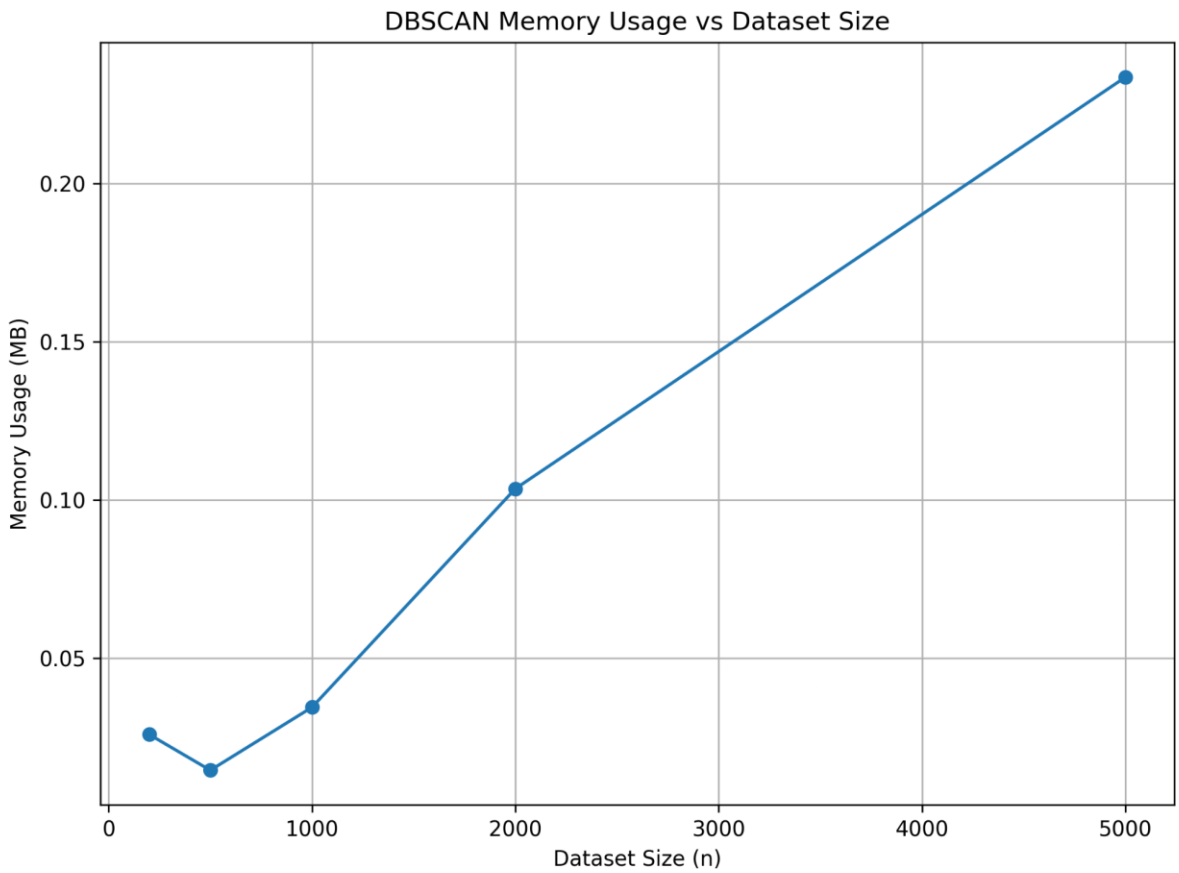


3. Memory Usage

Memory usage for DBSCAN increases as the dataset size (n) increases. As shown in the graph below DSCAN memory usage grows consistently from smaller datasets **(n = 100)** to larger datasets **(n = 5000)** demonstrating that DBSCAN requires more memory to process larger datasets.

The graph below has a dip around n = 500. This is likely due to the system's memory fluctuations and garbage collection rather than DBSCAN itself.

Despite the dip in the graph with smaller datasets we have found DBSCAN consistently uses more memory as the dataset size (n) increases.



4. DBSCAN Final Observations

DBSCAN is a solid choice for small and large clustering. It yielded great results on the smaller Iris dataset and the larger synthetic datasets thus demonstrating efficient clustering performance.

Despite DBSCAN having a higher runtime and inconsistent memory usage as the size of a given dataset increases it still produced high silhouette scores with the right parameters.

Overall, DBSCAN is a reliable choice when you are prioritizing accuracy with larger datasets. Recall that DBSCAN's performance is heavily influenced by the values of eps and min_samples, choose these parameters carefully.

Firefly:

The Firefly algorithm is the only nature inspired algorithm in this report. It was evaluated with both the synthetic and iris dataset to analyze memory usage, runtime, accuracy, and how increasing the data size affects these metrics.

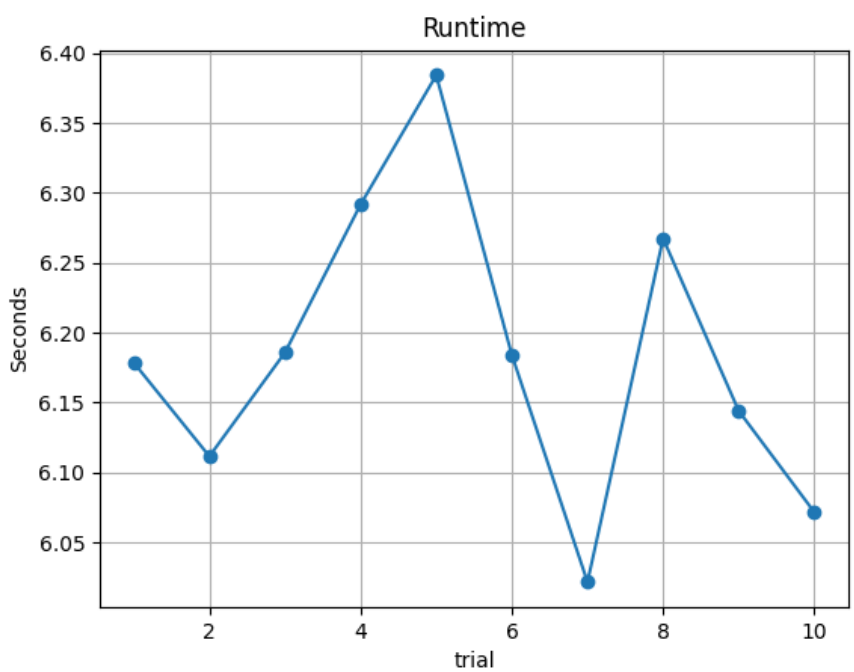
1. Runtime

The runtime of the firefly algorithm increased rapidly as the dataset size(n) increased. In the synthetic dataset graph, the runtime increased from about 25.59 seconds to n = 200 to about 756.51 seconds at n = 5000.

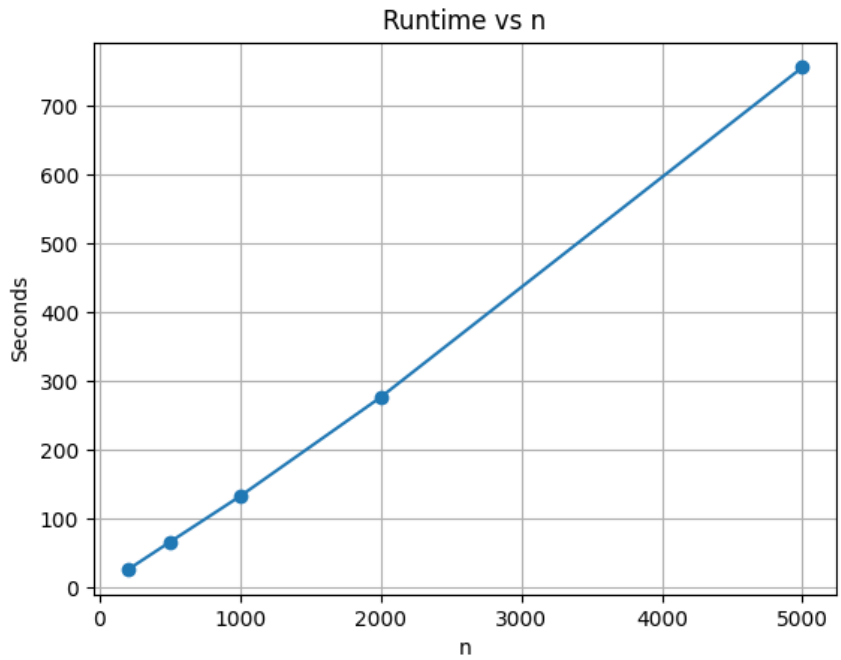
This is expected because each firefly has to be compared with the other fireflies. Because of this, the algorithm has a high computational cost.

As for the Iris dataset, the runtime stayed around 6 seconds per trial. Because the Iris dataset only has 150 data points the changes in runtime were small compared to the synthetic dataset which goes up to 5000.

Out of all the algorithms in this report, the firefly algorithm is significantly the slowest making it the worst option if runtime is the main priority.



Runtime of Firefly using the synthetic dataset



Runtime of Firefly using the synthetic dataset across different parameters

2. Accuracy (Cluster Quality)

The cluster quality was measured using the silhouette score and SSE. For the Iris dataset, the silhouette score changed between trials because Firefly uses randomness. So, some trials produce better clusters than others. The silhouette scores were around 0.06 to 0.49.

As for the synthetic dataset, the silhouette score was high overall. It was about 0.71 at n = 200, 0.84 at n = 500, then dropped around n = 2000, and increased again by n = 5000. So, it could still find decent clusters, but the quality was not consistent.

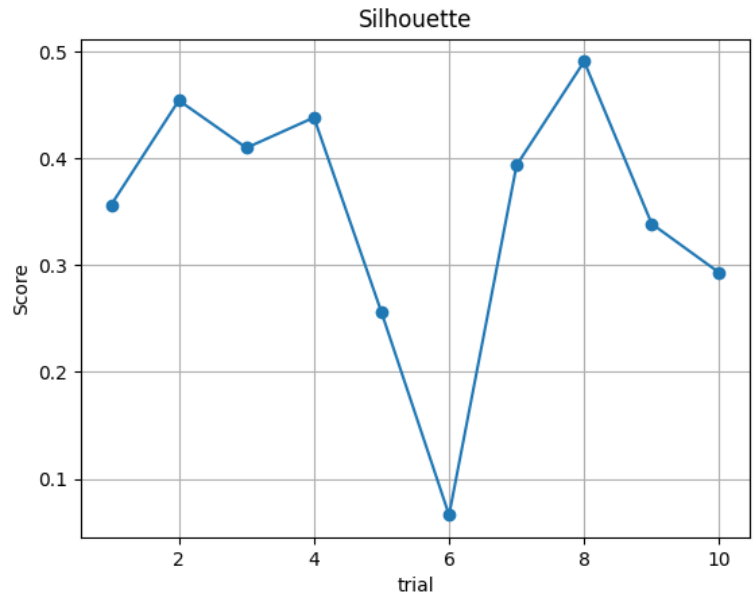
The SSE graph noticeably increased as the synthetic dataset size increased. This is expected because the larger datasets have more points, so the total error becomes larger. The SSE increased from about 4,243 at n = 200 to about 129,743 at n = 5000. While this is a very large increase, it does not mean that the Firefly algorithm is worse, it mostly just shows that total error grows when there are more data points.

1	algorithm	dataset	n	trial	runtime_seconds	sse	silhouette_score
2	Firefly	Iris	150	1	6.178466558456421	508.16988248042946	0.3567945685198302
3	Firefly	Iris	150	2	6.111464262008667	325.01964282952747	0.4545000028124127
4	Firefly	Iris	150	3	6.185765027999878	439.553934421014	0.41002989887481994
5	Firefly	Iris	150	4	6.2917094230651855	403.48640831156854	0.4384565165782748
6	Firefly	Iris	150	5	6.384149074554443	417.22929354653445	0.2559351275339185
7	Firefly	Iris	150	6	6.183948755264282	566.6144333650485	0.06597235408426923
8	Firefly	Iris	150	7	6.021525859832764	484.7905302174164	0.39383716356820164
9	Firefly	Iris	150	8	6.2673234939575195	410.2039964548692	0.49102345912972034
10	Firefly	Iris	150	9	6.143955945968628	452.4279769311274	0.3388455982917624
11	Firefly	Iris	150	10	6.071467876434326	568.1830202146747	0.29302614641388575

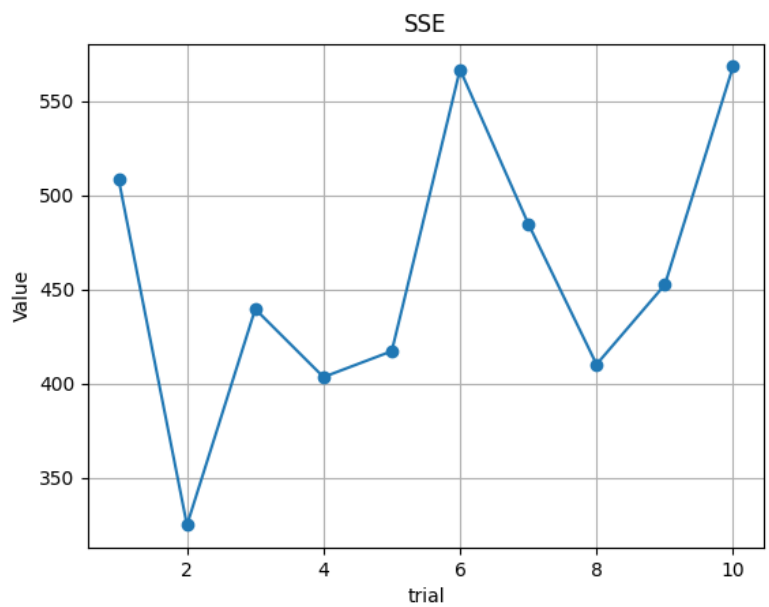
Firefly using the Iris dataset

1	algorithm	dataset	n	runtime_seconds	sse	silhouette_score
2	Firefly	Iris	150	6.184	457.5679	0.3498

Summary of Firefly using the Iris dataset



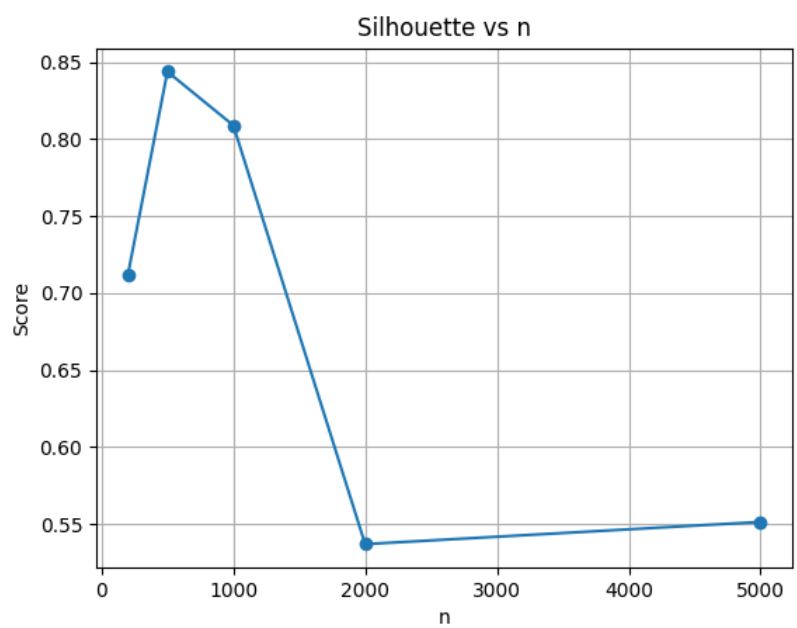
Silhouette of Firefly using the Iris dataset



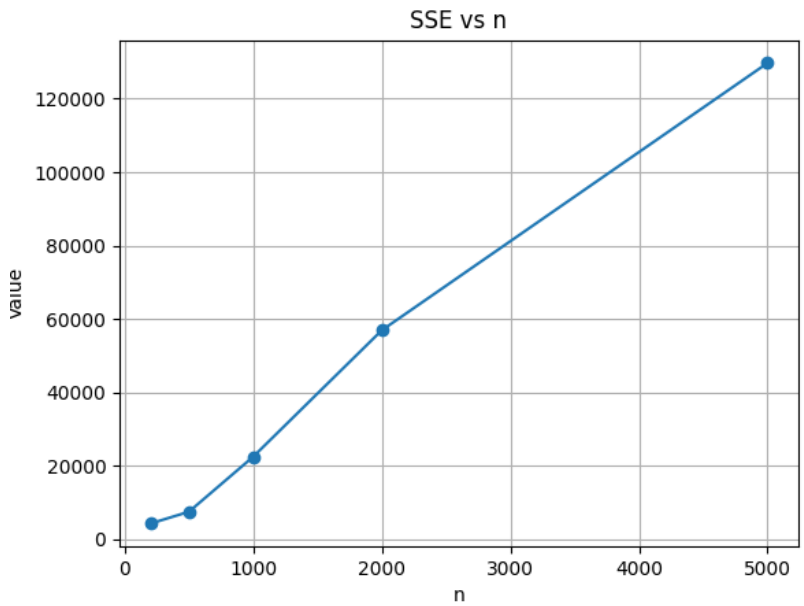
SSE of Firefly using the Iris dataset

1	algorithm	dataset	n	runtime_seconds	memory_mb	sse	silhouette_score
2	Firefly	Synthetic	200	25.5911	0.0307	4243.3452	0.7119
3	Firefly	Synthetic	500	65.9108	0.0663	7467.062	0.8438
4	Firefly	Synthetic	1000	132.6107	0.1298	22469.4127	0.8089
5	Firefly	Synthetic	2000	276.6231	0.2932	56910.7526	0.5371
6	Firefly	Synthetic	5000	756.5137	0.7002	129743.4572	0.5515

Firefly using the iris dataset across different parameters



Silhouette of Firefly using the iris dataset across different parameters

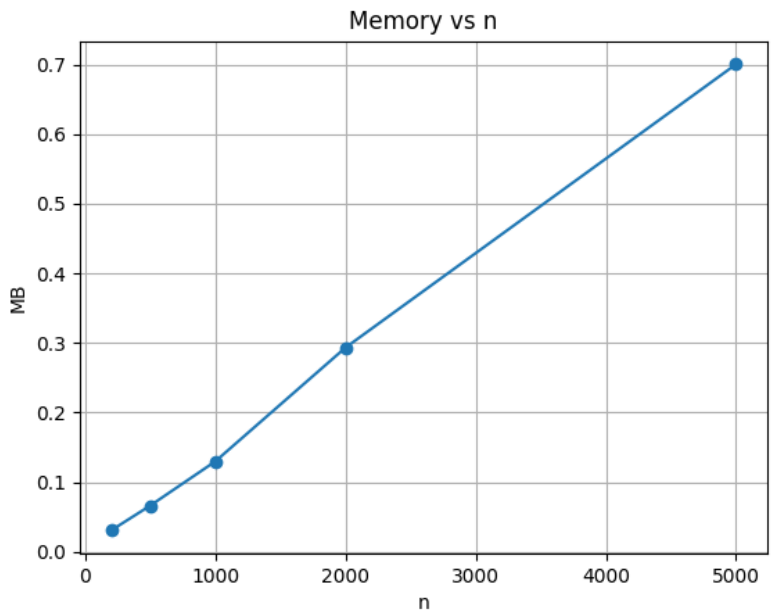


SSE of Firefly using the iris dataset across different parameters

3. Memory Usage

The memory usage mostly increased as the dataset size increased. For the synthetic dataset graph, the memory was lower for smaller datasets but reached about 0.70 MB at n = 5000. Larger datasets require more storage for data points, fireflies, and calculations.

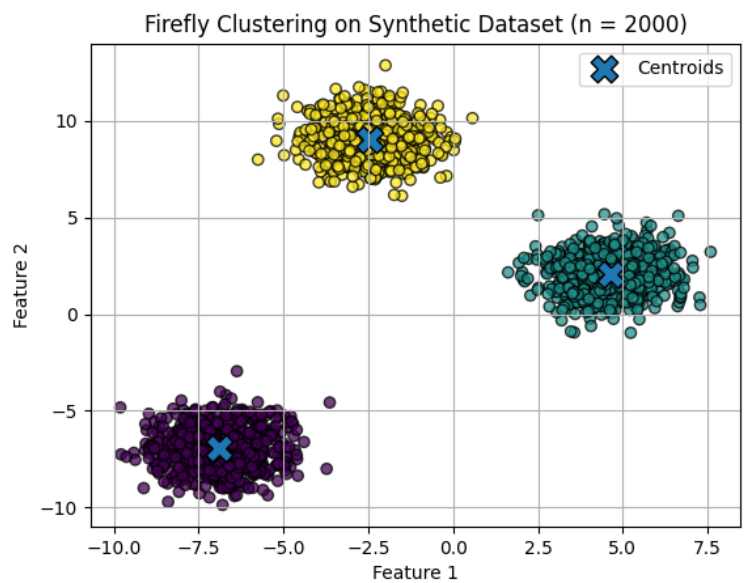
For the Iris dataset, the memory usage was very small and dropped close to zero after the first trial. This is most likely due to Python memory handling and garbage collection.



Memory of Firefly using the iris dataset across different parameters

4. Firefly Conclusion

The firefly algorithm provides consistently good clustering quality on smaller datasets. But because of its high computational cost, it does not scale well with large datasets, especially its runtime. The runtime becomes much higher as the dataset size increases. Because of this, the Firefly algorithm is a poor choice for larger datasets. It works best with smaller datasets.



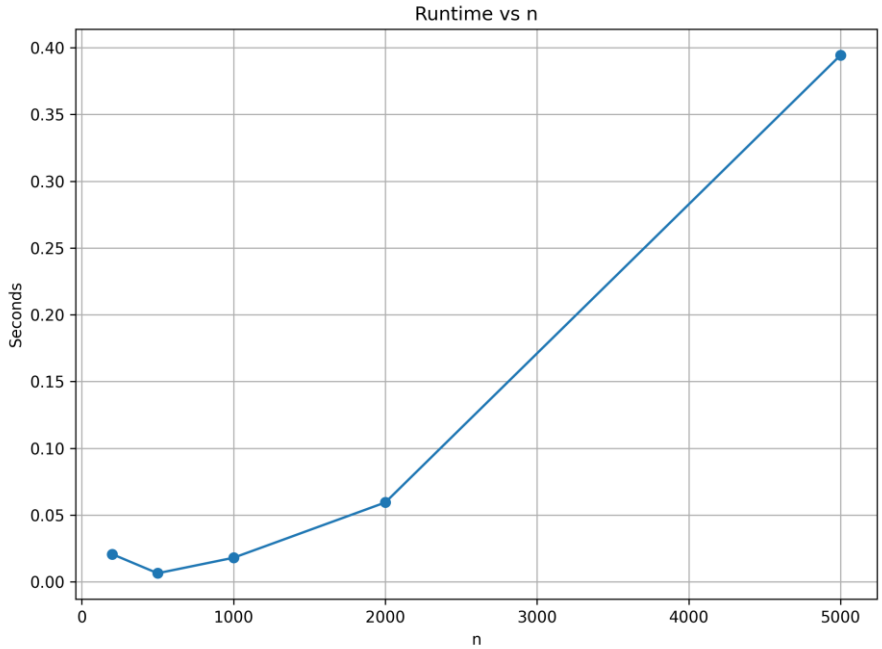
Hierarchical Clustering:

1. Runtime

The runtime of this algorithm is increased as the data set size (n) increased. In the synthetic dataset, the runtime started very low at about 0.0208 seconds at n = 200, and increased to about 0.3944 seconds at n = 500. This is expected because the

Hierarchical clustering requires computing and updating distance matrix, which becomes more expensive as dataset size increases.

As for the Iris data set, because the dataset is small (n = 150), the runtime stayed extremely small, averaging around 0.0005 to 0.0023 seconds per trial.



Runtime of Hierarchical using the iris dataset across different parameters

2. Accuracy (Cluster Quality)

The cluster quality was measured using the silhouette score and SSE. For the synthetic dataset, the silhouette score was consistently high, from about 0.8467 at n = 200 to about 0.8416 at n = 5000. This shows that even as the dataset increased the cluster separation remained strong still.

The SSE increased as dataset size increased, from about 364.47 at n = 200 to about 10059.17 at n = 5000. This is expected because larger datasets contain more data points, which means a higher total error value

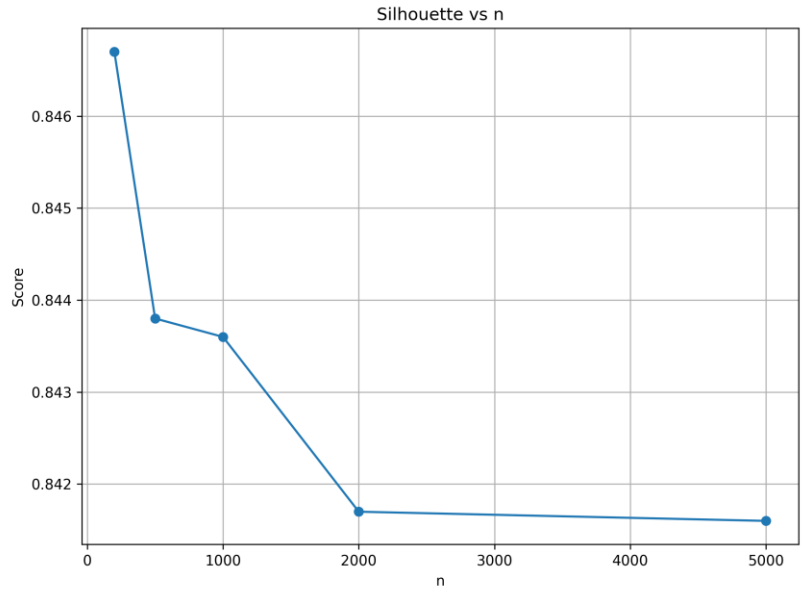
As for the Iris dataset, the silhouette score stayed constant at 0.4467 across all trials. This is because Hierarchical clustering is deterministic, meaning it produces the same clustering result every time. The SSE also stayed constant at about 148.88.

1	algorithm	dataset	n	trial	runtime_seconds	sse	silhouette_score
2	Hierarchical	Iris	150	1	0.01750499999616295	148.8762557795792	0.4466890410285909
3	Hierarchical	Iris	150	2	0.0006076999998185784	148.8762557795792	0.4466890410285909
4	Hierarchical	Iris	150	3	0.0005923999997321516	148.8762557795792	0.4466890410285909
5	Hierarchical	Iris	150	4	0.000662600010400638	148.8762557795792	0.4466890410285909
6	Hierarchical	Iris	150	5	0.0008613000100012869	148.8762557795792	0.4466890410285909
7	Hierarchical	Iris	150	6	0.0007105999975465238	148.8762557795792	0.4466890410285909
8	Hierarchical	Iris	150	7	0.0007048999977996573	148.8762557795792	0.4466890410285909
9	Hierarchical	Iris	150	8	0.0007741000008536503	148.8762557795792	0.4466890410285909
10	Hierarchical	Iris	150	9	0.0005328999977791682	148.8762557795792	0.4466890410285909
11	Hierarchical	Iris	150	10	0.000527900003362447	148.8762557795792	0.4466890410285909

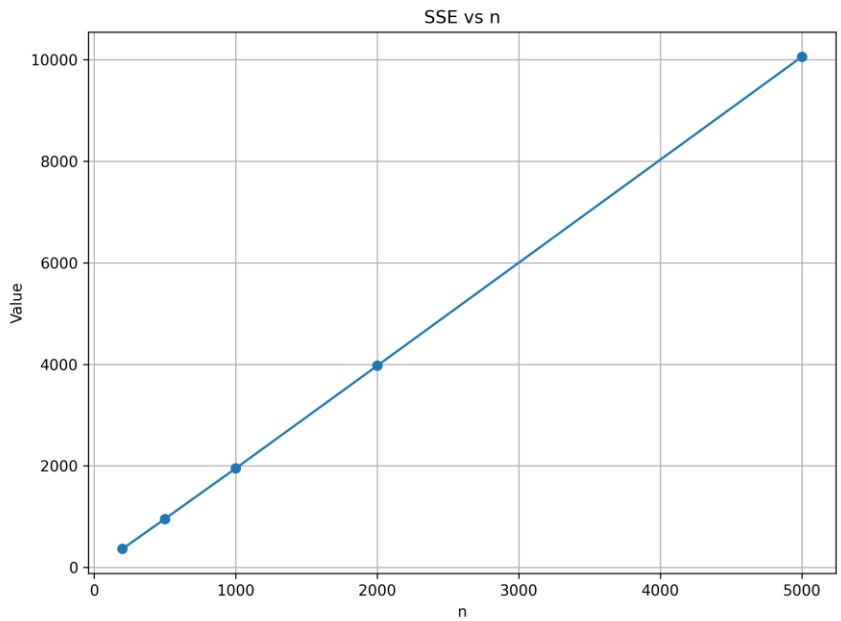
Hierarchical using the Iris dataset

1	algorithm	dataset	n	runtime_seconds	sse	silhouette_score
2	Hierarchical	Iris	150	0.0023	148.8763	0.4467

Summary of Hierarchical using Iris dataset



Silhouette of Hierarchical using the Iris dataset across different parameters



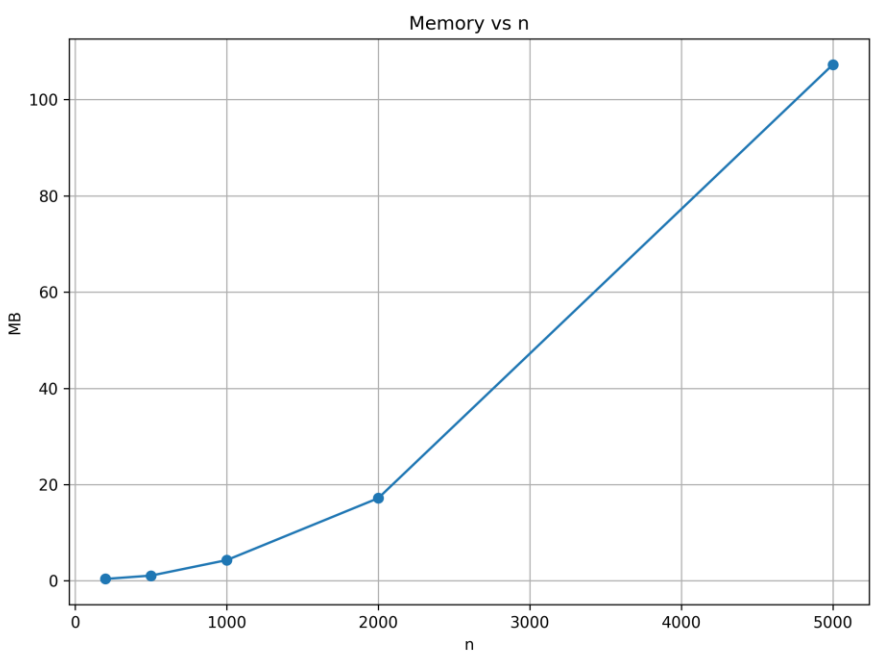
SSE of Hierarchical using the Iris dataset across different parameters

1	algorithm	dataset	n	runtime_seconds	memory_mb	sse	silhouette_score
2	Hierarchical	Synthetic	200	0.0208	0.3847	364.4733	0.8467
3	Hierarchical	Synthetic	500	0.0065	1.0747	955.6493	0.8438
4	Hierarchical	Synthetic	1000	0.0181	4.291	1950.8815	0.8436
5	Hierarchical	Synthetic	2000	0.0595	17.1613	3972.6166	0.8417
6	Hierarchical	Synthetic	5000	0.3944	107.2706	10059.1708	0.8416

Summary of Hierarchical using the iris dataset across different parameters

3. Memory Usage

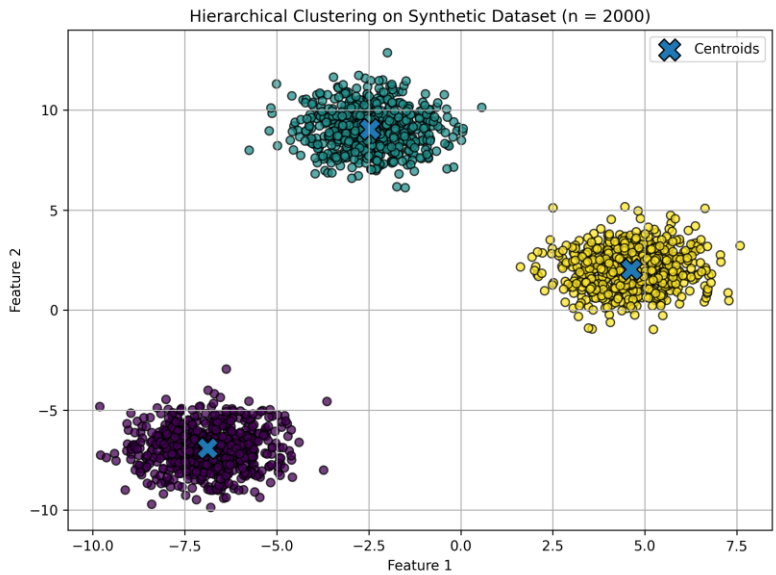
The memory increased significantly as the dataset size increased. For the synthetic dataset, the memory usage grew from about 0.38 MB at n = 200 to about 107.27 MB at n = 5000. This is expected because Hierarchical clustering requires storing a full distance matrix, which grows at a rate proportional to n squared. So, memory becomes a huge limitation for larger datasets.



Hierarchical using the iris dataset across different parameters

4. Hierarchical Clustering Conclusion

Hierarchical Clustering created strong clustering results, especially silhouette score. It did very well in terms of accuracy and had stable results due to its deterministic nature. But even though it has clustering quality, it does not scale well for large datasets. The results show that both runtime and memory usage increase significantly as the dataset size increases. Also, memory usage grows very quickly because it needs to store a full matrix. Because of this, Hierarchical clustering is best suited for smaller datasets where accuracy are more important than computational efficiency.



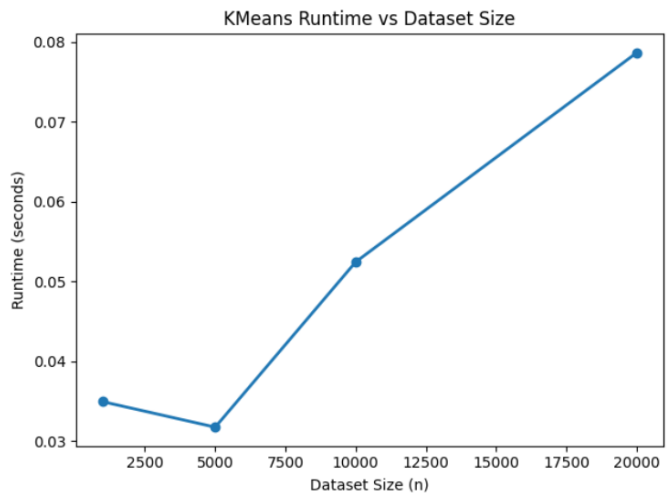
K-Means:

K-Means was evaluated using synthetic datasets of increasing size to analyze runtime, memory usage, clustering accuracy, and overall clustering performance. Synthetic datasets were generated using `make_blobs()` with three cluster centers, and K-Means was performed using `k = 3` clusters for consistency across all tests. To reduce Python startup effects and improve timing accuracy, each runtime measurement was averaged across multiple trials after an initial warm-up run.

1. Runtime

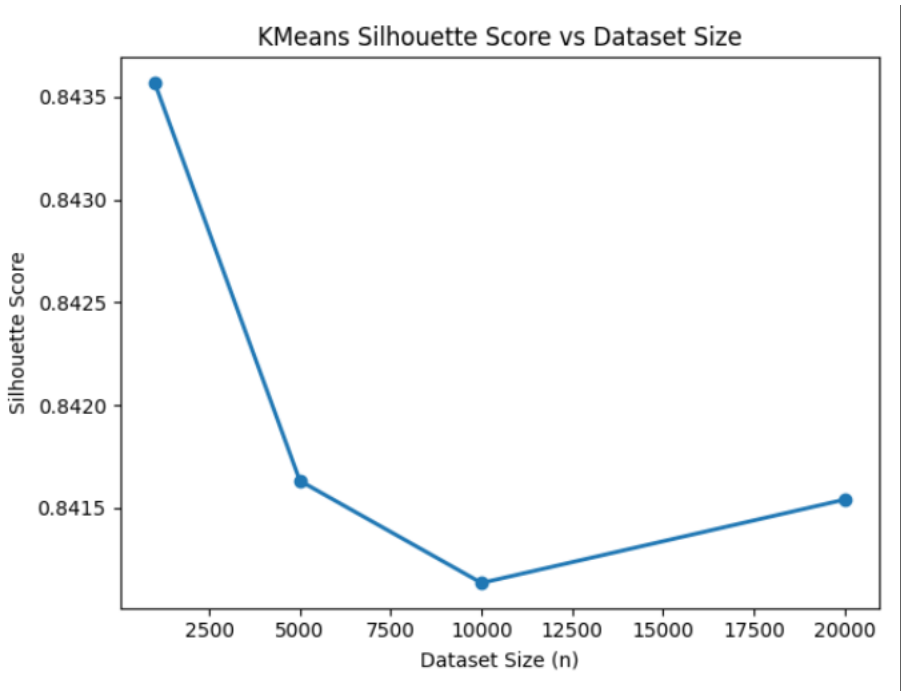
As expected, the runtime of K-Means generally increased as the dataset size (n) increased. For smaller datasets, slight fluctuations occurred due to the algorithm completing very quickly and system-level factors such as caching and background

processes affecting measurements. However, when using larger synthetic datasets (n = 1000 to n = 20000), the overall trend showed increasing runtime as the number of data points increased. This demonstrates that K-Means scales efficiently for moderate-to-large datasets and performs faster than more computationally expensive clustering methods such as DBSCAN or Hierarchical Clustering when runtime is a primary concern.



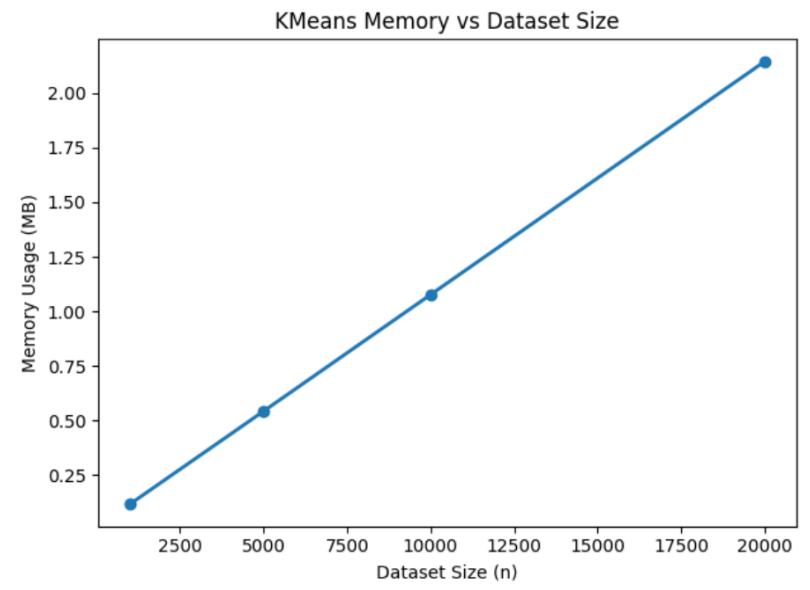
2. Accuracy (Cluster Quality)

Cluster quality was measured using the silhouette score. Across all synthetic dataset sizes, K-Means maintained consistently high silhouette scores around 0.84, indicating strong separation between clusters and good clustering performance. Unlike DBSCAN, which is highly sensitive to parameter choices such as `eps` and `min_samples`, K-Means produced stable results because the synthetic data was generated with clearly separated cluster centers. The slight variation in silhouette scores across dataset sizes is likely due to natural randomness in data generation rather than poor clustering performance. This demonstrates that K-Means performs very well when clusters are well-defined and relatively spherical in shape.



3. Memory Usage

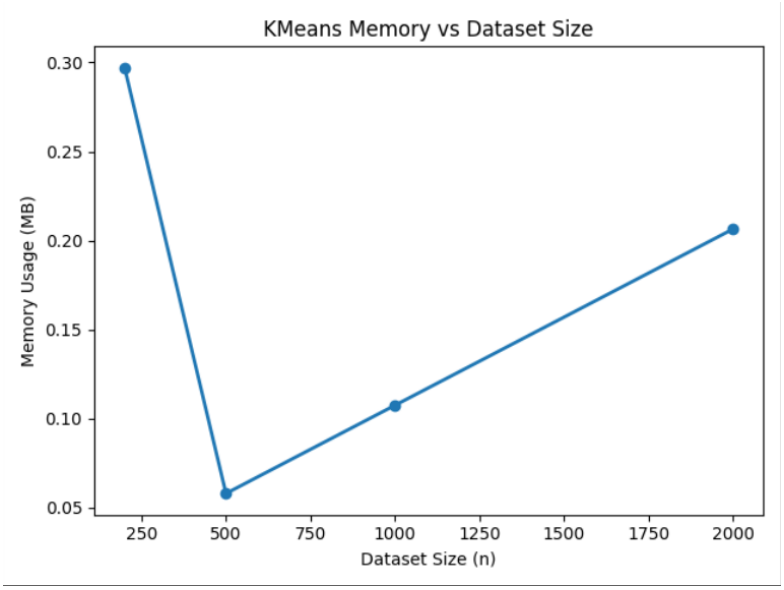
Memory usage increased consistently as the dataset size (n) increased. Starting with smaller datasets, K-Means required relatively little memory, but as the number of samples increased to 20,000, memory usage grew steadily to over 2 MB. This behavior is expected because K-Means must repeatedly calculate distances between points and centroids during each iteration. As dataset size increases, more memory is required to store these calculations and cluster assignments. Compared to DBSCAN, K-Means showed more predictable memory growth and remained computationally efficient even for larger datasets.



This is Synthetic Memory vs Datasize

```
PS C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\code> python k-meansgraphs.py
Current working directory: C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\code
Looking for CSV at: C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\results\kmeans_synthetic_results.csv
CSV loaded successfully.
  algorithm    dataset      n  k  runtime_seconds  memory_mb  silhouette_score
0    KMeans  Synthetic   1000  3         0.036079   0.117200          0.843571
1    KMeans  Synthetic   5000  3         0.031839   0.542865          0.841633
2    KMeans  Synthetic  10000  3         0.051125   1.076754          0.841136
3    KMeans  Synthetic  20000  3         0.089211   2.144754          0.841542
Saved: C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\graphs\kmeans_scaling_graphs\kmeans_runtime.png
Saved: C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\graphs\kmeans_scaling_graphs\kmeans_memory.png
Saved: C:\Users\Nevaeh\Downloads\DOA_Project-main\DOA_Project-main\graphs\kmeans_scaling_graphs\kmeans_silhouette.png
0 ⚠ 0
```

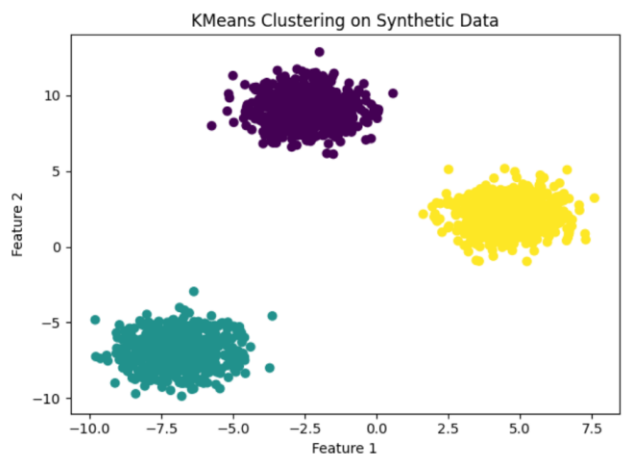
This is Synthetic Memory vs Data Size CSV



This is Iris memory vs datasize

4. K-Means Final Observations

K-Means proved to be a strong choice for clustering synthetic datasets with clearly separated groups. It demonstrated fast runtime, stable memory usage growth, and consistently high silhouette scores across all dataset sizes. One major advantage of K-Means is its simplicity and efficiency when the number of clusters is known in advance. However, unlike DBSCAN, K-Means requires the user to specify the number of clusters (`k`) beforehand and may perform poorly when clusters have irregular shapes or significant noise. Overall, K-Means is a reliable choice when prioritizing speed, scalability, and performance on well-structured datasets. For larger datasets with clear cluster boundaries, K-Means provides strong clustering quality with relatively low computational cost.



Interpret Results:

The empirical analysis shows clear differences between the algorithms for runtime, accuracy, and scalability. K-Means was the fastest and scaled the best as dataset size increased. Firefly and Hierarchical clustering had the worst runtime performance, especially for larger datasets.

For accuracy, DBSCAN, K-Means, and Hierarchical clustering had strong and consistent silhouette scores. While Firefly also had good results sometimes, but it was inconsistent because of its randomness since it is a natured inspired algorithm.

For memory usage, Hierarchical clustering used the most memory due to storing a distance matrix, while K-means was the best at memory efficiency. Firefly used moderate memory but was mainly limited by runtime.

Overall, K-Means is the best choice for large datasets because they are fast and scalable. DBSCAN is the best for accuracy and cluster shape. Hierarchical clustering is best used when the number of clusters are unknown, and Firefly is best used for complex, non-linear problems.

A limitation for this experiment is that synthetic data may not fully represent real-world data. Also, parameter choices can affect results. If we were to do this again, we would test more real datasets and try more parameter tuning.

For future research, we could explore optimizing nature-inspired algorithms like Firefly or testing performance on high-dimensional datasets.

Sources:

ChatGPT/Claude: Used to create some of the K-means, DBSCAN, Firefly, and Hierarchical algorithm files.

Wikipedia: https://en.wikipedia.org/wiki/K-means_clustering

Geeks for Geeks: https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/

https://github.com/firefly-cpp/FireflyAlgorithm

https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/

Team Roles:

Project Stage:	Names:	% of Total:
Brainstorming:	Jake Batton, Nevaeh Zumbrun, Patriya Murray	100%
Coding:	Jake Batton, Nevaeh Zumbrun, Patriya Murray	25%, 25%, 50% respectively
Experiment design:	Jake Batton	100%
Visualization/results:	Jake Batton, Nevaeh Zumbrun, Patriya Murray	25%, 25%, 50% respectfully

Communication: Jake Batton, Nevaeh Zumbrun, Patriya Murray 100%

Submission: Jake Batton, Nevaeh Zumbrun, Patriya Murray 100%

Report writing: Jake Batton, Nevaeh Zumbrun, Patriya Murray 33.33%, 33.33%, 33.33% respectfully

Presentation: Jake Batton 100%

Main Difficulties/Roadblocks (paragraph):

The main difficulties lay in time management. We wanted to reach our goal of having 4 algorithms run really big datasets that can also report to us their metrics, but our other classes made it difficult to spend plenty of time. Also, the firefly algorithm is not documented very well which made it incredibly difficult to code.